

Practica 7: Métodos de integración numérica



Materia: Sistemas de Control

Alumnos: Joshua Osorio

Profesor : Luara Jimenez Beristain

Fecha: Octubre/01/2025

Índice

Practica 7: Métodos de integración numérica.....	1
.....	1
Objetivo.....	2
Material y Equipo.....	2
Introducción.....	2
Parte I.....	2
Parte II.....	2
Parte III.....	3
Parte IV.....	5
Parte V.....	12
Parte VI.....	13
Parte VII.....	13
Parte VIII.....	14
Observaciones y Conclusiones:.....	15
Bibliografía.....	15

Objetivo

Aplicar algoritmos de integración numérica para resolver las ecuaciones diferenciales ordinarias y predecir el comportamiento de sistemas continuos ante distintas entradas y/o condiciones iniciales empleando las herramientas de Matlab y Simulink, con actitud analítica y crítica.

Material y Equipo

- **Plataforma de simulación:** MATLAB, Simulink
- **Comandos utilizados:** step, hold on, legend, pzmap, damp, sgrid, stepinfo

Introducción

Parte I

Para obtener una solución aproximada de la Ecuación Diferencial Ordinaria (EDO) en distintos instantes de tiempo, se recurre al uso de distintos algoritmos numéricos. Estos algoritmos, denominados métodos de integración de ecuaciones diferenciales se suelen implementar en distintas herramientas de software, algunas de propósito general como Matlab, Scilab, etc., o bien para dominios específicos (el PSpice por ejemplo para simular circuitos). A continuación, utilizaremos algunos métodos de integración numérica muy conocidos y exploraremos algunas de sus características. Como herramienta se utilizará primero el entorno gráfico de simulación que provee el software Simulink y luego Matlab.

Parte II

Método de Euler

Dado un sistema de la forma:

$$\dot{x}(t) = f(t, x)$$

Con condición inicial conocida $x(t) = \frac{dx}{dt}|_{p_0} = f(t_0, x_0)$ y $p_0 = (t_0, x_0)$ es el punto donde pasa la curva solución.

El método de Euler brindará una solución numérica según la fórmula:

$$x(t_{k+1}) = x(t_k) + h \cdot f(t_k, x(t_k))$$

Donde $h \cong t_{k+1} - t_k$ se denomina paso de integración. Normalmente se suelen obviar los argumentos $\diamond$ y se reescribe:

$$x_{k+1} = x_k + h \cdot f(t_k, x(t_k))$$

La idea geométrica del método de Euler se representa en la figura 1. Es importante notar que el valor de y_1 puede determinarse, ya que t_0, y_0 y $f(t, y)$ son datos del problema y h es un valor dado.

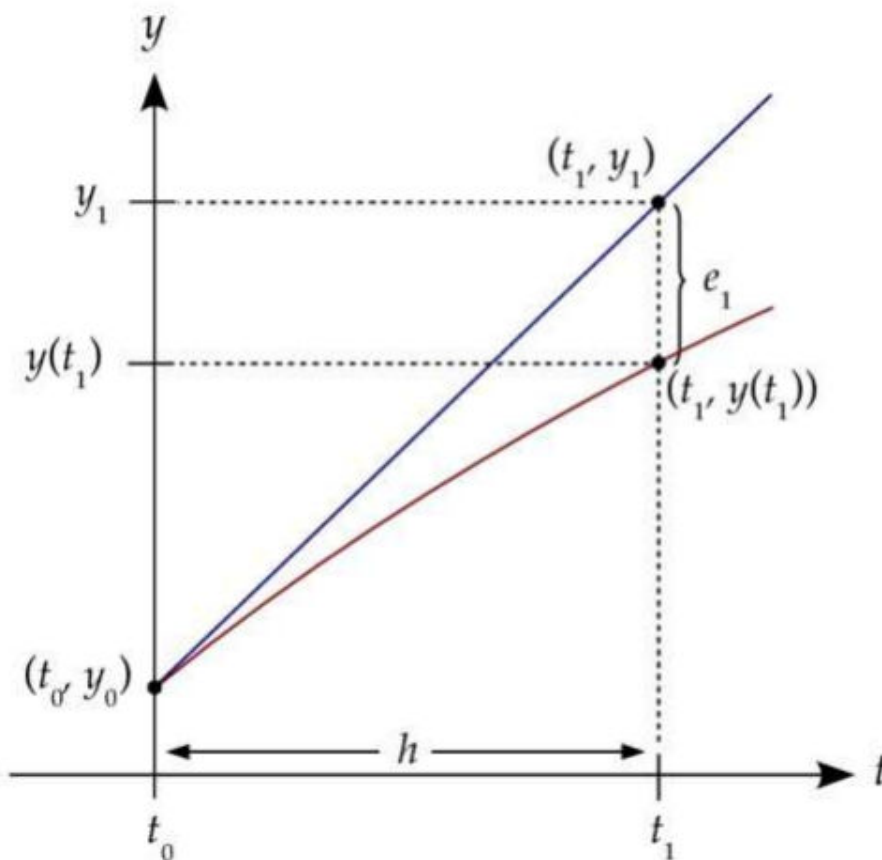


Figura 1. Interpretación geométrica del método de Euler.

El valor y_1 es la aproximación que proporciona el método de Euler al valor exacto de la solución en el nodo t_1 , esto es, y_1 es aproximadamente $y(t_1)$. Mientras que el error cometido al hacer dicha aproximación es $e_1 = |y(t_1) - y_1|$, verificando la propiedad de convergencia $\lim_{h \rightarrow 0} e(h) = 0$.

Parte III

Para la dinámica de un sistema de un circuito RL serie con $R=1\Omega$, $L=1H$ y una fuente de alimentación de voltaje de 1V en directa, la ecuación diferencial ordinaria está representada por:

$$\dot{x}(t) = v_s - \frac{R}{L} x = x(t)$$

si consideramos una condición inicial nula $x(0) = 0$, podemos resolver fácilmente la ecuación, obteniendo la solución analítica:

$$x_a(t) = 1 - e^{-t}$$

Teniendo en cuenta que $f(t_k, x_k) = 1 - x_k$, el método de Euler calculará la secuencia de valores según:

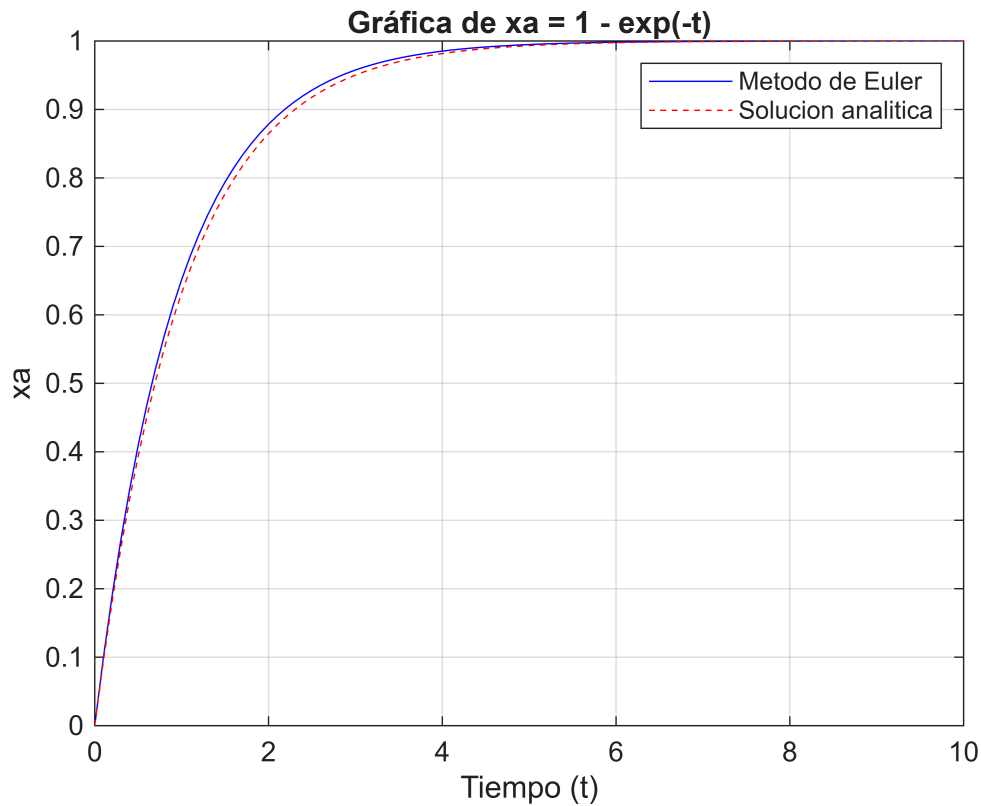
$$x_{k+1} = x_k + h \cdot (1 - x_k)$$

Los cálculos normalmente se realizan utilizando un programa en una computadora. En Matlab, la función que calcula esta solución puede programarse como sigue:

```
function [t,x]=funcionEuler(h)
    % Resuelve dx/dt = 1 - x
    % Entre t = 0 y t = 10 para x0 = 0
    % Utilizando el método de Euler
    x = 0;
    t = 0;
    N = 10/h;
    for k = 1:N
        t(k+1) = t(k) + h;
        x(k+1) = x(k) + h*(1 - x(k));
    end
end
```

Luego, invocando desde la línea de comando podemos obtener la solución brindada por el método de Euler, de forma que la variable $\diamond$ contenga el tiempo y $\diamond$ los valores de la solución y además podemos ver graficada la solución.

```
figure Name 'Ejemplo 1';
[t,x]=funcionEuler(0.1);
plot(t,x, "b");
hold on;
xa = 1 - exp(-t);
plot(t, xa, 'r--');
legend('Metodo de Euler', 'Solucion analitica');
title('Gráfica de  $x_a = 1 - \exp(-t)$ ');
xlabel('Tiempo (t)');
ylabel('xa');
grid on;
```



Parte IV

1.- Programar la rutina que resuelve con Euler el sistema del circuito RL:

$$\dot{x}(t) = 1 - x(t)$$

2.- Obtener la solución para los casos de integración $h = 0.01$, $h = 0.1$ y $h = 1$ y comparar cada solución con la solución analítica graficando el error. No olvide utilizar el comando "legend" para identificar cada caso.

```
figure Name 'Parte IV 1 h = 0.01';
subplot(2,1,1);
[t,x]=funcionEuler(0.01);
plot(t,x,'b');

hold on;
xa = 1 - exp(-t);
plot(t, xa,"r--");

title("Grafica para h = 0.01");
legend("Metodo n merico", "Metodo anal tico");
xlabel("Tiempo (s)");
ylabel("Var dependiente");

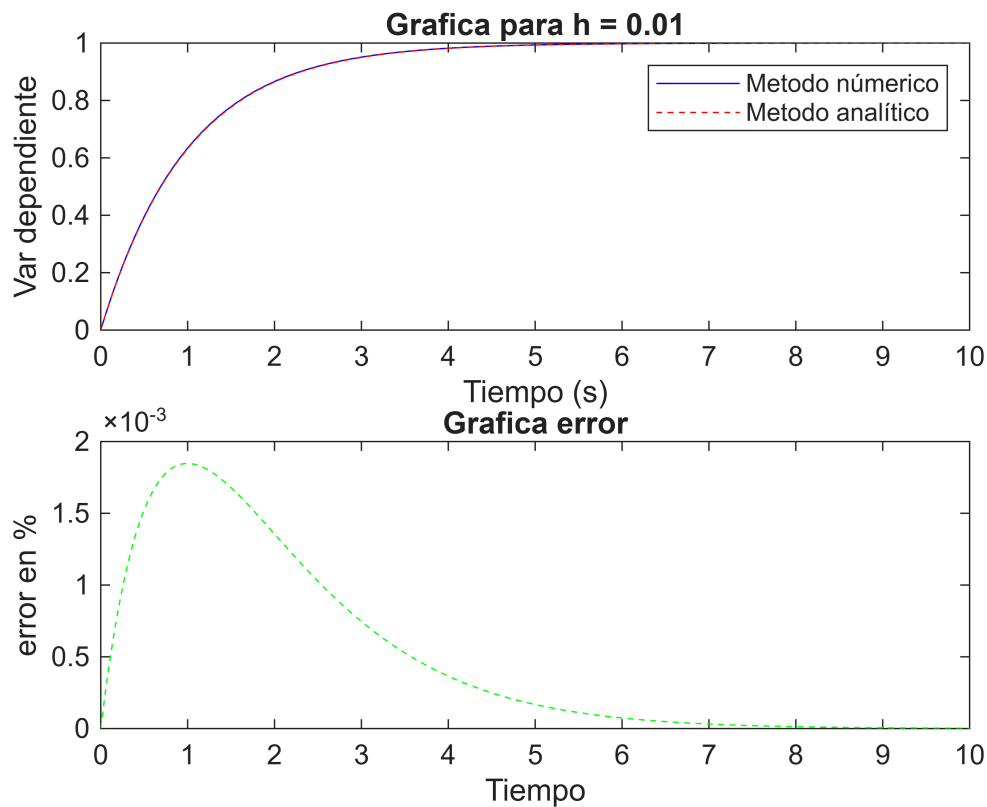
subplot(2,1,2);
```

```

error = abs(x-xa);
plot(t, error, 'g--');

title("Grafica error");
xlabel("Tiempo");
ylabel("error en %");

```



```

figure Name 'Parte IV 2 h = 0.1';
subplot(2,1,1);
[t,x]=funcionEuler(0.1);
plot(t,x,'b');
hold on;

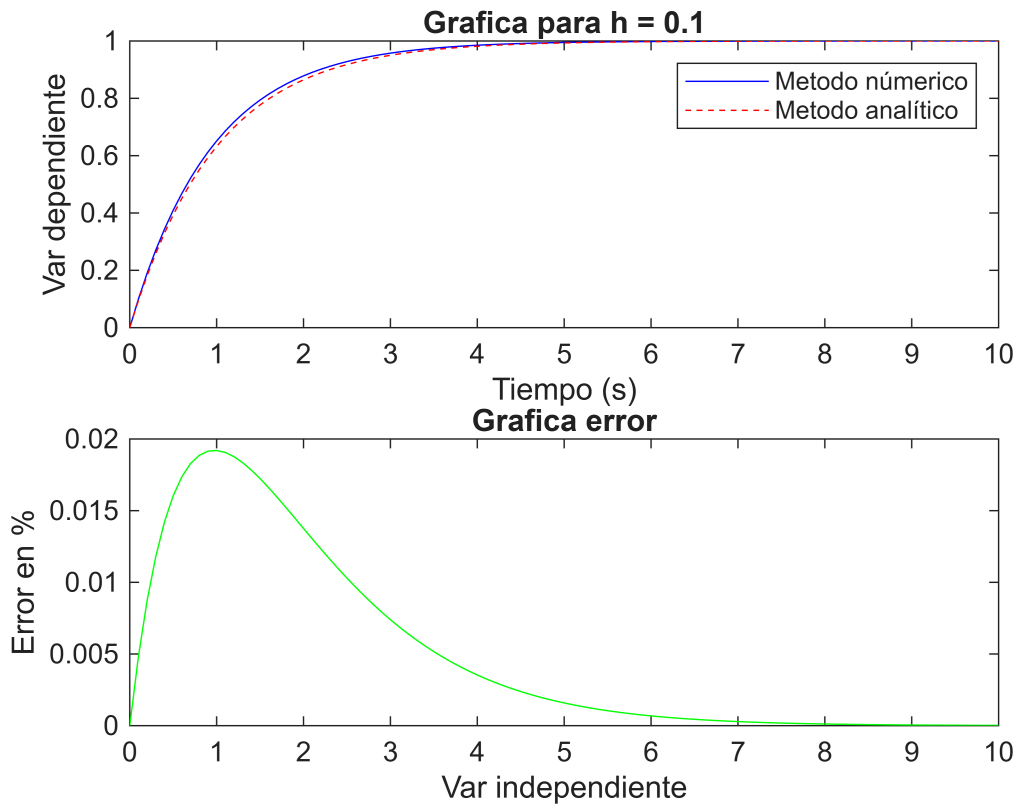
xa = 1 - exp(-t);
plot(t, xa, 'r--');

title("Grafica para h = 0.1");
legend("Metodo n merico", "Metodo anal tico");
xlabel("Tiempo (s)");
ylabel("Var dependiente");

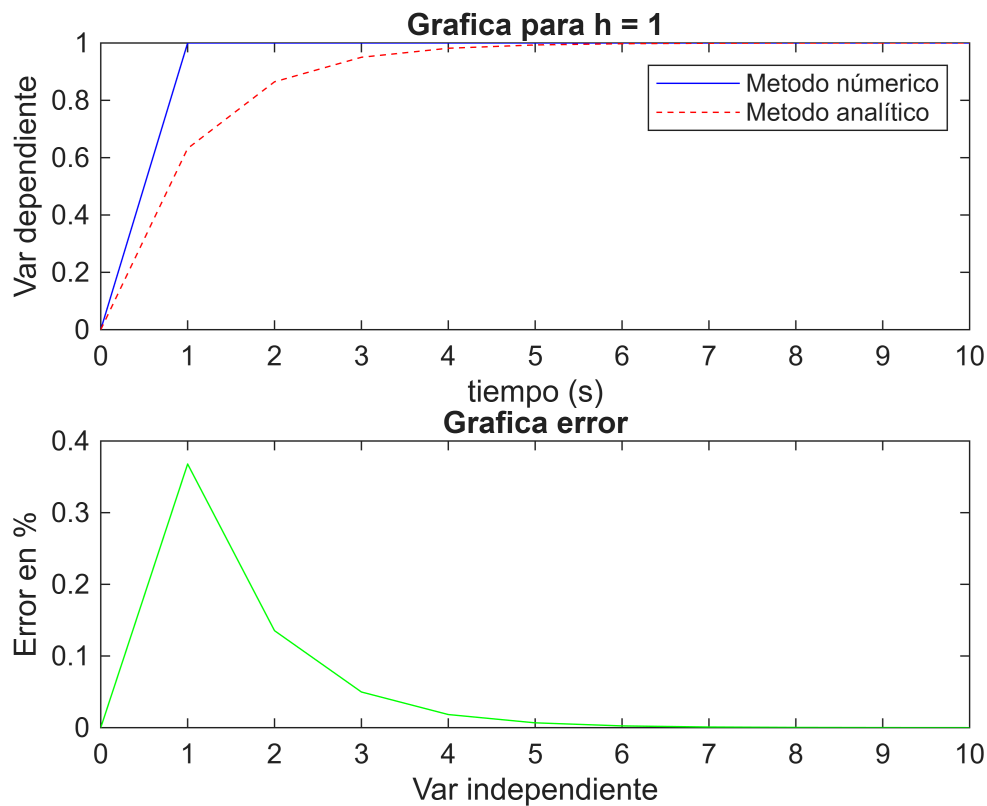
subplot(2,1,2);
error = abs(x-xa);
plot(t, error, 'g');
title("Grafica error");
ylabel("Error en %");

```

```
xlabel("Var independiente");
```



```
figure Name 'Parte IV 2 h = 1';  
subplot(2,1,1);  
[t,x]=funcionEuler(1);  
plot(t,x,'b');  
hold on;  
  
xa = 1 - exp(-t);  
plot(t, xa,'r--');  
  
title("Grafica para h = 1");  
legend("Metodo n merico", "Metodo anal tico");  
xlabel("tiempo (s)");  
ylabel("Var dependiente");  
  
subplot(2,1,2);  
error = abs(x-xa);  
plot(t, error,'g');  
title("Grafica error");  
ylabel("Error en %");  
xlabel("Var independiente");
```



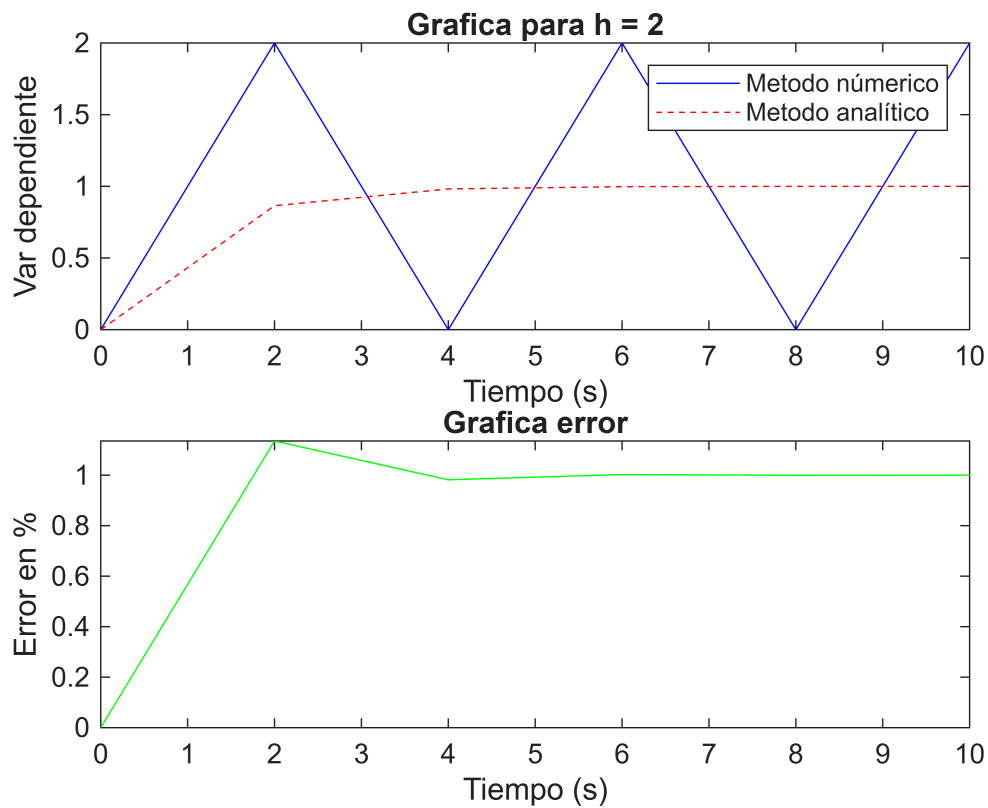
3.- Analizar que pasa cuando se hace $h = 2$ y $h = 3$. Explique su respuesta.

```
figure Name 'Parte IV 3 h = 2';
subplot(2,1,1);
[t,x]=funcionEuler(2);
plot(t,x,'b');
hold on;

xa = 1 - exp(-t);
plot(t, xa,'r--');

title("Grafica para h = 2");
legend("Metodo numérico", "Metodo analítico");
xlabel("Tiempo (s)");
ylabel("Var dependiente");

subplot(2,1,2);
error = abs(x-xa);
plot(t, error,'g');
title("Grafica error");
ylabel("Error en %");
xlabel("Tiempo (s)");
```

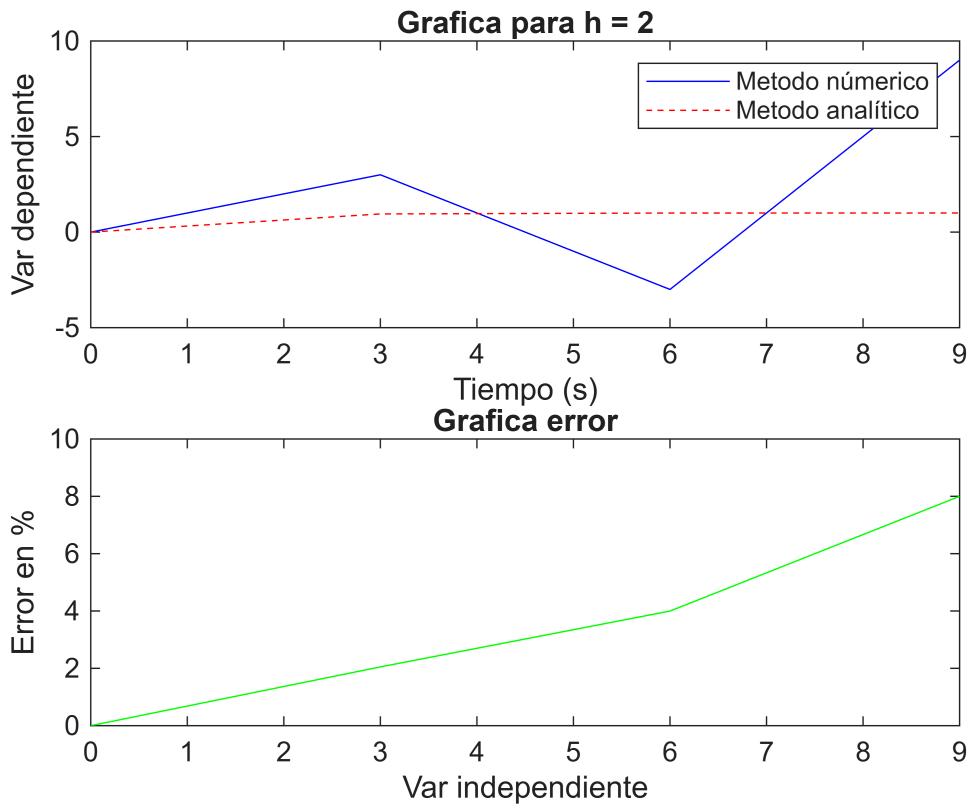



```
figure Name 'Parte IV 3 h = 3';
subplot(2,1,1);
[t,x]=funcionEuler(3);
plot(t,x,'b');
hold on;

xa = 1 - exp(-t);
plot(t, xa,'r--');

title("Grafica para h = 2");
legend("Metodo numérico", "Metodo analítico");
xlabel("Tiempo (s)");
ylabel("Var dependiente");

subplot(2,1,2);
error = abs(x-xa);
plot(t, error,'g');
title("Grafica error");
ylabel("Error en %");
xlabel("Var independiente");
```



4.- Realiza ajustes en el código anterior y considera a $R=2K\Omega$, $L=100mH$ y modifica la fuente de alimentación a una amplitud de 5V. Registre sus resultados.

$$\dot{x}(t) = v_s - \frac{R}{L} x = x(t)$$

tenemos que adecuar a la ecuacion.

```
function [t,x]=funcionEuler2(h)
% Resuelve dx/dt = 1 - x
% Entre t = 0 y t = 10 para x0 = 0
% Utilizando el método de Euler
V = 5; % Es el voltaje v_s
R= 2000; % Valor de la resistencia
L= 0.1; % Valor de la bobina.
x = 0;
t = 0;
N = 10/h;
for k = 1:N
    t(k+1) = t(k) + h;
    x(k+1) = x(k) + h*(V - (R/L).*x(k));
end
end
```

Despues probamos con el ejercicio 1.1

```
figure Name 'Parte IV 1 h = 0.01';
subplot(2,1,1);
[t,x]=funcionEuler2(1);
plot(t,x,'b');

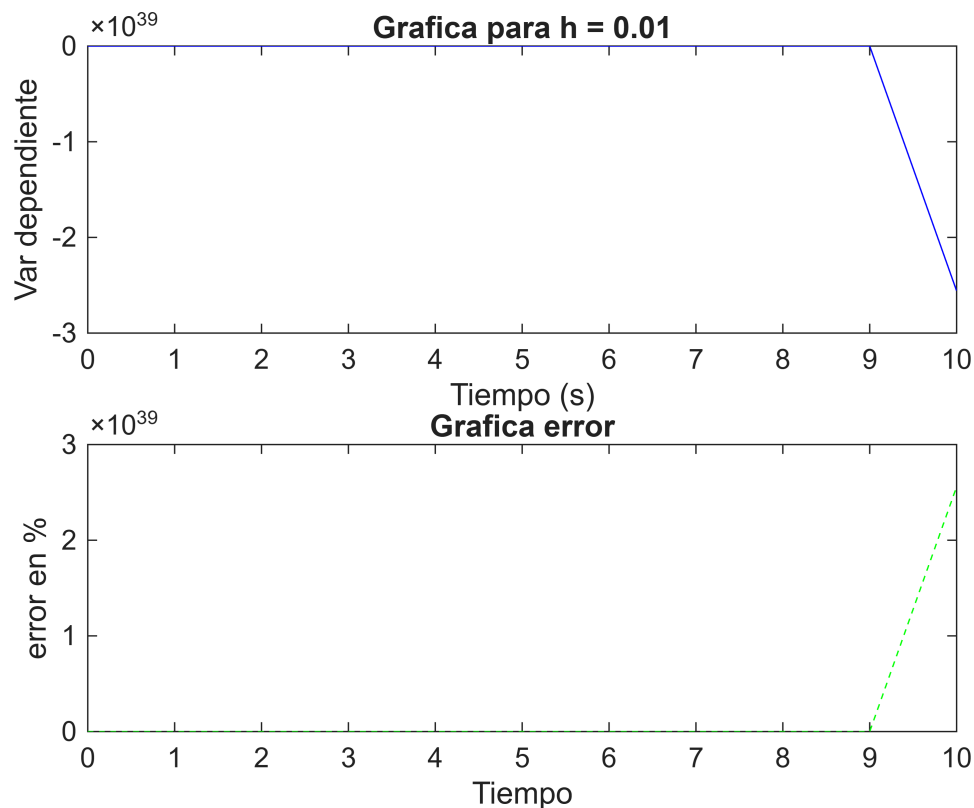
hold on;
xa = 5 - (2000/0.1).* exp(-t);
%plot(t, xa,"r--");

title("Grafica para h = 0.01");
%legend("Metodo n meroico", "Metodo anal tico");
xlabel("Tiempo (s)");
ylabel("Var dependiente");

subplot(2,1,2);

error = abs(x-xa);
plot(t, error,'g--');

title("Grafica error");
xlabel("Tiempo");
ylabel("error en %");
```



5.- Reporta tus resultados al profesor

Parte V

La siguiente rutina de Matlab implementa el método de Euler de manera genérica:

```
function [t,x] = euler1(f,x0,tf,h)
    % resuelve dx / dt = f(t,x)
    %Entre t=0 y t =tf para x(0) = x0
    %Utilizando el método de Euler
    x = x0;
    t = 0;
    N = tf/h;
    for k=1:N
        t(k+1)=t(k)+h;
        x(:,k+1)=x(:,k) +h*f(t(k), x(:,k));
    end
end
```

Generalmente, las rutinas que implementan los distintos métodos numéricos se programan de manera genérica, y los sistemas se programan en rutinas separadas. Para probar esta rutina, se utilizará un sistema de ecuaciones diferenciales de un péndulo con fricción:

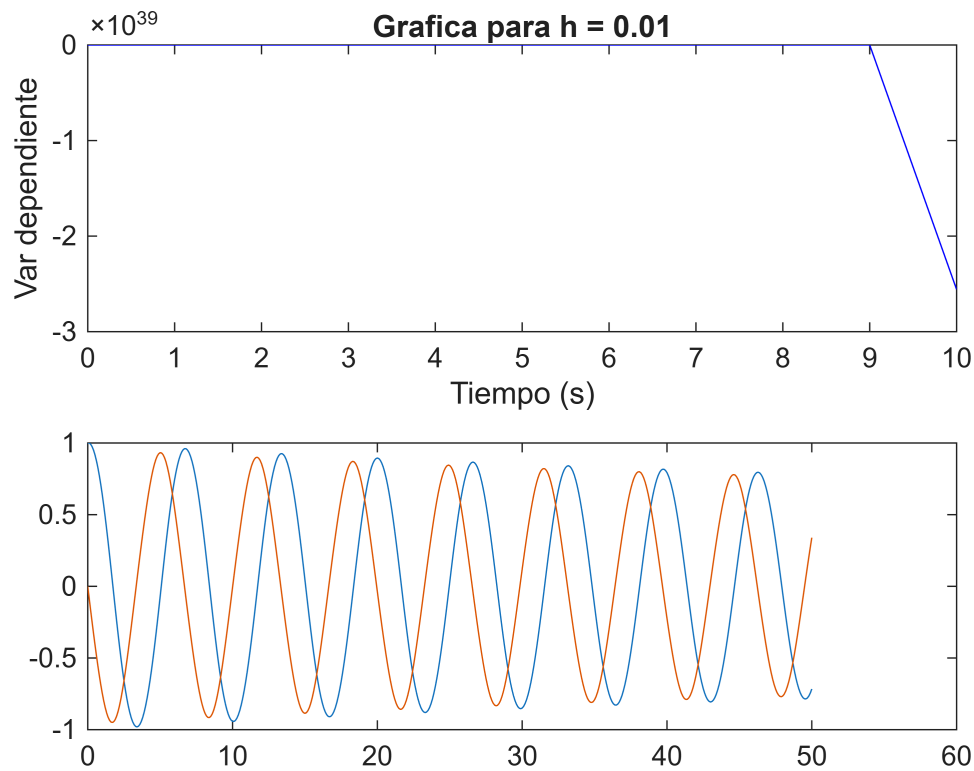
$$\begin{aligned}\dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\sin(x_1) - 0.1 \cdot x_2,\end{aligned}$$

el cual, se deberá programar como sigue:

```
function dx=pendulo(t,x)
    dx = [x(2); -sin(x(1))-0.1*x(2)];
end
```

Luego, para simular esta ecuación desde la condición inicial $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ podemos invocar desde Matlab:

```
[t,x] = euler1(@pendulo, [1;0], 50, 0.1);
plot(t,x);
```



Parte VI

Se pide entonces:

1. Programar la rutina de Euler y la función que calcula la ecuación del péndulo.
2. Simular con pasos de integración $= 0.01$, $= 0.1$ y $= 1$.
3. Reporta tus resultados al profesor.

Parte VII

La ecuación del péndulo se puede representar en Simulink mediante diagrama a bloques como se muestra en la Figura 2. En dicho diagrama a bloques pueden verse además los bloques To Workspace que nos permitirán graficar los resultados de simulación en Matlab. Alternativamente pueden utilizarse bloques Scope para visualizar los resultados directamente en Simulink.

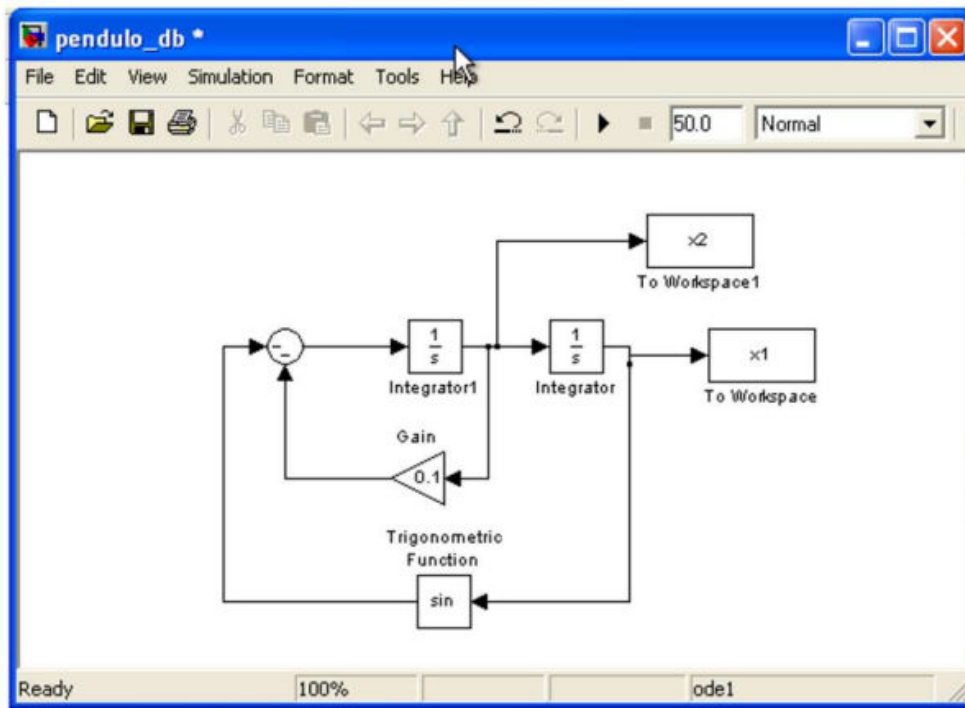


Figura 2. Diagrama de bloques en Simulink del péndulo.

Para elegir los parámetros de simulación en tanto, se debe seleccionar la opción correspondiente, lo que abre la ventana de la Figura 3. En dicha figura, los parámetros fueron elegidos para utilizar el método de Euler, simulando con un paso fijo de 0.1 hasta un tiempo final de 50.

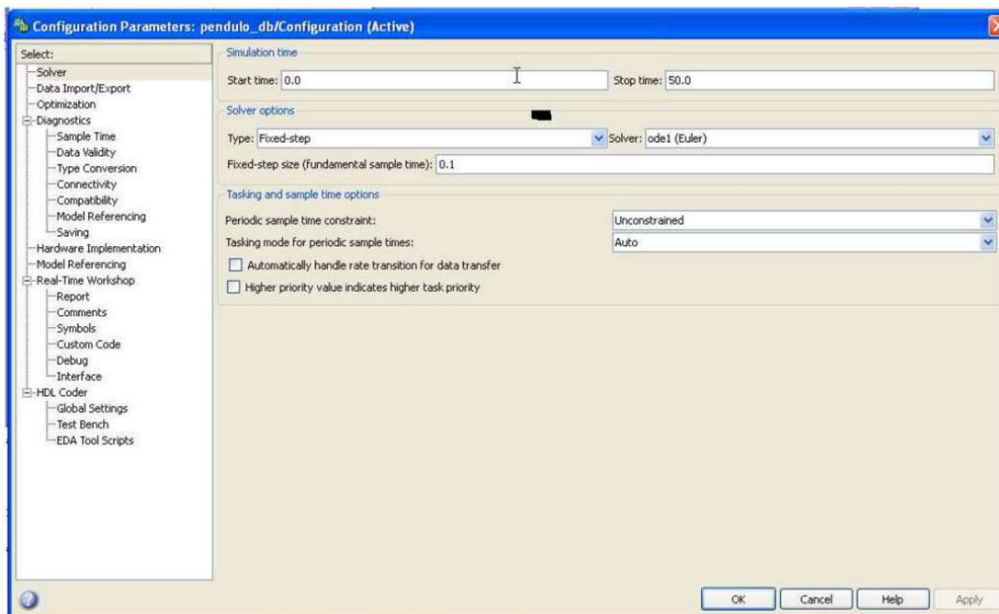


Figura 3. Ventana de parámetros de Simulink.

Parte VIII

Se pide entonces:

1. Construir el diagrama de bloques del péndulo y simular con paso de integración $= 0.1$ el sistema utilizando el método de Euler y comparar con el obtenido en el punto VI.
2. Reporta tus resultados al profesor.

Observaciones y Conclusiones:

Investigar los métodos de integración numérica Euler y Heun para resolver ecuaciones diferenciales. Incluye explicación grafica

Bibliografía

- [1] "plot - Gráfica de líneas en 2D - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/plot.html?overload=plot+false&websocket=on>
- [2] "dsolve - Solve system of differential equations - MATLAB." Available: <https://la.mathworks.com/help/symbolic/dsolve.html?websocket=on>
- [3] "fplot - Representar una expresión o función - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/fplot.html?overload=fplot+false&websocket=on>
- [4] "ezplot - (Not recommended) Easy-to-use function plotter - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/ezplot.html?overload=ezplot+false&websocket=on>
- [5] "subplot - Crear ejes en posiciones segmentadas - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/subplot.html?websocket=on>
- [6] "inline - (Not recommended) Construct inline object - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/inline.html?overload=inline+false&websocket=on>
- [7] "ode45 - Resolver ecuaciones diferenciales no rígidas; método de orden intermedio - MATLAB." Available: <https://la.mathworks.com/help/matlab/ref/ode45.html?websocket=on>
- [8] Clases Debbie (Dra. D), "EDO de segundo orden + Matlab," *YouTube*. Nov. 22, 2020. Available: <https://www.youtube.com/watch?v=JSN4XCVnzpo>
- [9] "Introducción al MATLAB Live Editor," *MATLAB*. Available: <https://la.mathworks.com/videos/introduccion-al-matlab-live-editor-119615.html>